whitenoise — User Guide

Stochastic White Noise Analysis (SWNA) — Hida–Bernido Framework

Package version 0.1.0 · Python ≥3.9

1. Overview

whitenoise is a research package for analysing the memory and diffusion properties of sequential observational data using the Stochastic White Noise Analysis (SWNA) with Memory framework developed by Bernido & Carpio-Bernido.

The central object of SWNA is the *Mean Square Displacement* (MSD), which characterises how the spread of a fluctuating observable grows with lag time Δ:

```
MSD(Δ) = (1 / (N−Δ)) · ∑[x(i+Δ) − x(i)]²    (summed over i = 0 … N−Δ−1)
```

Fitting a theoretical model to the empirical MSD yields the **memory parameter μ**, which quantifies how strongly the system's past fluctuations influence its future. The table below summarises the four diffusion regimes:

Regime	Criterion	Physical interpretation
Subdiffusive	$\mu < 0.95$	Past fluctuations resist future change — slow spreading
Near-Brownian	$0.95 \le \mu \le 1.05$	Weak or no memory — ordinary random walk
Superdiffusive	$1.05 < \mu \le 2.0$	Past fluctuations reinforce future ones — fast spreading
Hyperballistic	$\mu > 2.0$	Extreme persistence — strongly correlated dynamics

> **Scope:** The package accepts any sequential 1D observable, not just time series. Column 1 is the independent variable (time, index, lag, distance …); column 2 is the observable. The word "time series" is used loosely throughout.

2. Installation

```
pip install -e .          # development install (editable, from the repo root)
pip install whitenoise     # future: once published to PyPI
```

Dependencies: numpy ≥1.20, scipy ≥1.7, matplotlib ≥3.4, pandas ≥1.3. Optional: openpyxl (for Excel export).

```
import whitenoise as wn   # standard import alias used throughout this guide
```

3. CSV Input Format

The package reads **2-column CSV files** with a required header row. The header encodes column names and physical units in the format `name [unit]`:

```
time [months], sunspot_number [count]
1, 12.4
2, 15.1
3, 14.8
```

Rule	Detail
Column 1	Independent variable (time, index, frequency, distance, ...). Always.
Column 2	Observable to analyse. Always.
Header format	`name [unit]` — space before bracket is optional: `co2[ppm]` also works.
Unitless column	Use empty brackets: `flux []` or `value [unitless]`
No brackets	Also accepted: `time, distance` — unit defaults to empty string.
Multi-column CSV	Column 1 = independent variable; all other columns = separate observables (batch mode).

Automatic validation warnings (non-fatal)

On read, the package checks for four conditions and prints a ⚠ warning if triggered. None of them halt execution.

Code	Condition	Warning message fragment
V1	Column 1 values not strictly increasing	"not strictly increasing"
V2	Column 1 name is not a recognised independent-variable name (e.g., 'measurement')	"Column 1 name"
V3	Unit contains 'count'/'freq' and observable has negative values	"Value range"
V4	Column 1 spread is enormous relative to mean (possible column swap)	"Column order"

Reading a CSV

```
time, values, meta = wn.read_csv('sunspot.csv')

# meta keys:
```

```
#   x_name, x_unit, x_label   ← column 1
#   y_name, y_unit, y_label   ← column 2
#   source_file, n_points

print(meta['y_label'])    # e.g. 'sunspot_number (count)'
print(meta['x_label'])    # e.g. 'time (months)'
```

Reading a multi-column CSV

```
results = wn.read_csv_multi('climate.csv')
# Returns list of (time, values, meta) tuples — one per observable column.
print(len(results))       # e.g. 3  (co2, temperature, sst)
```

4. The Full Pipeline

`CSV file` → `read_csv()` → `detrend() / normalize()` → `analyze()` → `AnalysisResult` → `.summary() / plot / export`

4.1 Quickstart — CSV path

```
result = wn.analyze('sunspot.csv', model='exponential')
result.summary()                # prints formatted parameter table
wn.plot_diagnostics(result)     # 2×2 overview: time series, MSD, PDF, parameters
```

4.2 Quickstart — NumPy array

```
import numpy as np
data = np.loadtxt('raw_data.txt')
result = wn.analyze(data, model='cosine', label='My System')
result.summary()
```

> When passing an array, `label=` is required — it becomes the dataset name in all outputs and plots. When passing a CSV path, the dataset name is taken from the filename (e.g. `'sunspot.csv'` → `'sunspot'`).

4.3 analyze() parameters

Parameter	Default	Description
`source`	—	CSV file path *or* 1D array-like.
`model`	`'cosine'`	Model name. See Section 6.

`label`	`''`	Dataset name (required for array input; auto-set from y_name for CSV).
`detrend_method`	`None`	Remove trend before fitting. `None` skips. Options: `'linear'`, `'polynomial'`, `'mean'`, `'moving_average'`.
`normalize`	`False`	Z-score normalise values before fitting.
`max_lag_fraction`	`1.0`	Fraction of lags used in fitting (1.0 = all N/2 lags).
`verbose`	`True`	Print ✓ progress lines and final R^2 / regime summary.
`fit_kwargs`	`{}`	Extra keyword arguments forwarded to `fit_msd()`.

4.4 AnalysisResult fields

Attribute	Type	Description
`.dataset_name`	str	Filename stem or label.
`.model`	str	Model name used.
`.fit`	FitResult or None	`None` if fitting failed (e.g. all-zero MSD).
`.lags`	ndarray	Lag array (integer steps).
`.msd_empirical`	ndarray	Empirical MSD values.
`.values`	ndarray	Preprocessed observable (after detrend/normalize if applied).
`.time`	ndarray	Independent variable array.
`.metadata`	dict	Keys: x_name, x_unit, x_label, y_name, y_unit, y_label, source_file, n_points.
`.regime`	str (property)	`'subdiffusive'` / `'near-Brownian'` / `'superdiffusive'` / `'hyperballistic'`
`.mu`	float (property)	Memory parameter μ (or H for fBm model).

5. Optional Preprocessing

Preprocessing functions are **never called automatically** by `analyze()`. The researcher applies them manually before analysis when scientifically appropriate.

> **When to detrend:** Only if the data has a deterministic trend that is not part of the stochastic dynamics you want to study (e.g. a linear warming trend in temperature data). Detrending changes the data; use `detrend_method=None` (the default) when in doubt.

5.1 detrend()

Method	Description
`'linear'`	Subtract best-fit straight line (degree-1 polynomial).
`'polynomial'`	Subtract degree-*n* polynomial fit. Set `poly_order=n`.
`'mean'`	Subtract global mean only.
`'moving_average'`	Subtract a moving-average trend. Set `window=k`.
`'psd'`	Bernido detrending §3.3 — power spectral density subtraction.
`'fingerprint'`	Bernido detrending §3.4 — stochastic fingerprint method.

```
fluct = wn.detrend(values, method='linear')
fluct = wn.detrend(values, method='polynomial', poly_order=2)
fluct = wn.detrend(values, method='moving_average', window=11)
```

5.2 normalize()

Method	Output range	Use case
`'zscore'`	mean=0, std=1	Standard normalisation before fitting.
`'minmax'`	[0, 1]	Bounded positive data.
`'mean'`	mean=1	Preserve scale, center around 1.
`'maxabs'`	[−1, 1]	Oscillating data; preserves zero and sign.

```
normed = wn.normalize(values, method='zscore')
normed = wn.normalize(values, method='maxabs')   # good for oscillating data
```

5.3 smooth()

```
smoothed = wn.smooth(values, window=7, method='moving_average')
smoothed = wn.smooth(values, window=7, method='gaussian')
```

5.4 Full preprocessing workflow example

```
time, values, meta = wn.read_csv('co2_keeling.csv')

# Inspect raw data first
wn.plot_series(time, values, metadata=meta)

# Remove quadratic trend (CO2 has a nonlinear growth trend)
fluct = wn.detrend(values, method='polynomial', poly_order=2)
wn.plot_series(time, fluct, metadata=meta, title='CO2 fluctuations')
```

```
# Analyse the residuals
result = wn.analyze(fluct, model='cosine', label='CO2 fluctuations')
result.summary()
```

6. Models

Run `wn.list_models()` to print a full table at any time. The researcher **always selects the model manually** — the package never auto-selects. If $R^2 < 0.5$ after fitting, the package prints a suggestion to try other models.

6.1 Available models

Name	Row	Parameters	MSD formula (schematic)	Reference
`cosine`	10	μ, ν	$\sqrt{\pi} \cdot \Gamma(\mu) \cdot \cos(\nu T/2) \cdot J_{\mu-1/2}(\nu T/2) / (T/\nu)^{1/2-\mu}$	Table 2.1 row 10
`exponential`	4	μ, β	$\Gamma(\mu) \cdot \beta^{-\mu} \cdot T^{\mu-1} \cdot e^{-\beta/T}$	Table 2.1 row 4
`sine`	9	μ, ν	Same structure as cosine but with $\sin(\nu T/2)$	Table 2.1 row 9
`fbm`	1	H	T^{2H} (H = 0.5 → Brownian)	Table 2.1 row 1
`sin_half`	2	(none)	$1 - \cos(T)$	Table 2.1 row 2
`cos_half`	3	(none)	$1 - J_0(T)$	Table 2.1 row 3
`inc_gamma`	14	ν, μ	Incomplete gamma function form	Table 2.1 row 14
`dna`	—	a, b, c	Plateau model: $MSD = a \cdot (1 - e^{-bT^c})$ [saturation at a]	DNA/polymer mechanics extension
`bessel_j0_cos`	—	μ, ν	Bessel J_0 × cosine modulation	Extension
`bessel_jmu_nu`	—	μ, ν	Power-law Bessel modulation	Extension

6.2 Stub models (not yet implemented)

The following are registered but raise `NotImplementedError` when called:

`exp_whittaker` · `bessel_K` · `hypergeom_F1` · `bessel_I` · `hypergeom_3F2` · `csc_power` · `cot_power` · `bessel_pair` · `bessel_pair2`

6.3 Parameters reference

Symbol	Name	Models	Interpretation
μ	Memory parameter	cosine, exponential, sine, and most others	Controls diffusion regime; see Section 1.
ν	Characteristic frequency	cosine, sine, bessel_jmu_nu, bessel_j0_cos	Sets the oscillation period of the modulating function.

β	Exponential decay rate	exponential	Controls how fast the memory decays with lag.
H	Hurst exponent	fbm	H < 0.5 subdiffusive; H = 0.5 Brownian; H > 0.5 superdiffusive.
a, b, c	Plateau parameters	dna	a = plateau MSD; b = rate; c = shape exponent.
N	Normalization scalar	all models	Fitted alongside physical params; scales MSD amplitude without changing shape.

6.4 Model selection guidance

Data type	Suggested starting model
Astronomical / geophysical time series	`cosine` or `exponential`
Earthquake / seismic data	`exponential`
X-ray binary / stellar flux	`cosine`
CO&sub2; / climate data	`cosine`
Brownian-like diffusion	`fbm` (H ≈ 0.5)
DNA / polymer conformation data	`dna`
Unknown / exploratory	Use `wn.batch_model_search()` to try all available models

7. FitResult — Fit Output

`result.fit` is a `FitResult` dataclass, or `None` if fitting failed. All fits use Levenberg–Marquardt via `scipy.optimize.curve_fit`. 95% confidence intervals are always included.

Attribute	Type	Description
`.params`	dict	Point estimates, e.g. `{'mu': 1.15, 'beta': 0.08, 'N': 2.3}`
`.std_errors`	dict	Standard errors (sqrt of pcov diagonal)
`.confidence_intervals`	dict	95% CI as `(lo, hi)` tuples; CI = param ± 1.96 × SE
`.r_squared`	float	R^2 = 1 – SS_res / SS_tot. Closer to 1 is better.
`.lags_used`	ndarray	Lags actually used in fitting (after max_lag_fraction truncation)
`.msd_fitted`	ndarray	N · msd_model(T) evaluated at lags_used
`.model`	str	Model name

```
result.fit.params           # {'mu': 1.15, 'beta': 0.08, 'N': 2.3}
result.fit.r_squared        # 0.9823
result.fit.confidence_intervals['mu']   # (1.11, 1.19)
print(result.fit.summary()) # formatted text block
```

R^2 interpretation: ≥0.8 = good fit; 0.5–0.8 = moderate; <0.5 = poor (the package will suggest alternative models automatically).

Fitting method — OLS, not WLS: The fitting uses ordinary least squares (OLS) via `scipy.optimize.curve_fit`, which minimises the unweighted sum of squared residuals (SSR). This treats all lag points equally, but MSD estimates at large lags are computed from fewer data pairs and are therefore noisier and less reliable than short-lag estimates. A weighted least squares (WLS) approach — down-weighting large lags in proportion to their variance — is not currently implemented.

Practical consequence: If the large-lag end of the empirical MSD is erratic, it can pull the fitted curve away from the well-sampled short-lag region, inflating or deflating R^2 in a misleading way. Use `max_lag_fraction=0.5` to restrict fitting to the first half of lags (the most statistically reliable region) when this occurs.

8. Plotting

Two plotting modules are provided. Both return a `matplotlib.figure.Figure` and accept `show=True` (default) and `ax=None` (for embedding into existing figures).

8.1 explore.py — Exploratory / interactive

Use these at any stage of the pipeline to quickly inspect data.

Function	When to use	Inputs
`wn.plot_series(x, values, metadata=meta)`	After `read_csv()` — inspect raw or detrended data	x array, values array, optional metadata dict
`wn.plot_msd_empirical(lags, msd, metadata=meta)`	After `compute_msd()` — inspect MSD shape before fitting	lags and msd arrays from `compute_msd()`
`wn.plot_msd(result)`	After `analyze()` — empirical scatter + fitted curve	AnalysisResult
`wn.plot_pdf(result)`	After `analyze()` — displacement histogram + Gaussian PDF	AnalysisResult; optional `lag_index=`
`wn.plot_timeseries(result)`	After `analyze()` — preprocessed observable	AnalysisResult
`wn.plot_diagnostics(result)`	Full 2×2 overview: time series, MSD, PDF, parameter box	AnalysisResult

8.2 publish.py — Publication-quality

Serif font, inward ticks, dpi=150, no emoji, clean styling. Use for figures in papers.

Function	Output
`wn.publish_msd(result, palette='default', save_path='fig.pdf')`	MSD plot: empirical scatter + fitted line, R^2 in legend
`wn.publish_pdf(result, lag_index=None, save_path='fig.pdf')`	Displacement PDF at a selected lag
`wn.publish_comparison(cr, save_path='fig.pdf')`	Horizontal bar chart of μ values with 95% CI error bars

```
# Save a publication-quality MSD figure as PNG
wn.publish_msd(result, save_path='figures/msd_sunspot.png')

# Save a comparison chart
wn.publish_comparison(cr, save_path='figures/comparison.pdf')
```

```
# Colorblind-friendly palette
wn.publish_msd(result, palette='colorblind', save_path='fig.pdf')
```

Supported save formats: `pdf`, `png`, `svg`, `eps`, `jpg`. Format inferred from the file extension.

8.3 Plotting at every pipeline stage — example

```
time, values, meta = wn.read_csv('earthquake_ph.csv')

# Stage 1: raw data
wn.plot_series(time, values, metadata=meta)

# Stage 2: after detrending
fluct = wn.detrend(values, method='linear')
wn.plot_series(time, fluct, metadata=meta, title='Residuals')

# Stage 3: empirical MSD before fitting
lags, msd = wn.compute_msd(fluct)
wn.plot_msd_empirical(lags, msd, metadata=meta)

# Stage 4: full analysis with fitted MSD
result = wn.analyze(fluct, model='exponential', label='Earthquake PH')
wn.plot_msd(result)          # empirical + fitted
wn.plot_diagnostics(result)  # full overview
```

9. Comparing Multiple Datasets

9.1 compare() — same model, multiple datasets

```
# Dict input — custom labels, mix of CSV paths and arrays
cr = wn.compare({
    'Earthquakes': 'earthquake_ph.csv',
    'Sunspots':    'sunspot.csv',
    'CO2 Keeling': co2_array,
}, model='exponential')

# List input — labels auto-set from filenames
cr = wn.compare(['chile.csv', 'japan.csv', 'ph.csv'], model='cosine')

# Per-dataset models
cr = wn.compare({'A': 'a.csv', 'B': 'b.csv'},
                model={'A': 'cosine', 'B': 'exponential'})
```

`cr` is a `ComparisonResult`:

```
cr.summary_df           # pandas DataFrame with one row per dataset
cr.results              # list of AnalysisResult objects
wn.print_comparison(cr) # print ASCII table to terminal
wn.publish_comparison(cr, save_path='comparison.pdf')
```

9.2 batch_analyze() — three input modes

Mode	source argument	Example
A — Folder	Folder path string	`wn.batch_analyze('data/xray_binaries/')`
B — List	List of CSV paths	`wn.batch_analyze(['a.csv', 'b.csv'])`
C — Multi-column CSV	CSV path with multiple observables	`wn.batch_analyze('climate.csv')`

```
cr = wn.batch_analyze('data/xray_binaries/', model='cosine', n_jobs=4)
cr.summary_df       # one row per dataset
wn.export_summary(cr, 'results/batch_summary.csv')
```

> **Parallel mode:** `n_jobs > 1` uses `ThreadPoolExecutor`. Works for CSV-path inputs only. For in-memory arrays, use `n_jobs=1`.

9.3 batch_model_search() — find the best model

```
cr = wn.batch_model_search(
    'sunspot.csv',
    models=['cosine', 'exponential', 'sine', 'fbm'],  # None = all available
)
# Prints R² for each model and reports the best one.
cr.summary_df.sort_values('r_squared', ascending=False)
```

9.4 ComparisonResult fields

Attribute	Description
`.summary_df`	DataFrame with columns: dataset_name, model, mu, mu_ci, nu_or_beta, nu_or_beta_ci, N, r_squared, regime.
`.results`	List of AnalysisResult objects (only successful fits).
`.models_used`	List of model names used.

10. Exporting Results

Function	Exports
`wn.export_csv(result, 'out.csv')`	Single AnalysisResult → one-row CSV with all parameters.
`wn.export_summary(cr, 'out.csv')`	ComparisonResult.summary_df → multi-row CSV.

```
result = wn.analyze('sunspot.csv', model='exponential')
wn.export_csv(result, 'results/sunspot_fit.csv')

cr = wn.compare(['a.csv', 'b.csv'], model='cosine')
wn.export_summary(cr, 'results/comparison.csv')
```

11. Advanced Usage

11.1 Direct MSD computation

```
lags, msd = wn.compute_msd(values)
lags, msd = wn.compute_msd(values, max_lag=100)        # cap at 100 lags
lags, msd = wn.compute_msd(values, normalize=True)     # divide by msd[0]
```

> The maximum lag is always capped at N/2 (where N is the number of data points). Beyond N/2, each MSD estimate uses fewer than N/2 pairs and becomes statistically unreliable.

11.2 Direct fitting

```
fit = wn.fit_msd(lags, msd_empirical, model='cosine')
fit = wn.fit_msd(lags, msd_empirical, model='cosine',
                 p0=[1.2, 0.01],          # initial guess [mu, nu]
                 bounds=([0.01, 1e-9], [5.0, 10.0]),
                 max_lag_fraction=0.5)    # fit only first half of lags
```

11.3 Model registry

```
wn.list_models()             # print all 19 models and their status
info = wn.get_model('cosine')
print(info['params'])        # ['mu', 'nu']
print(info['status'])        # 'available'
msd_val = info['msd'](10.0, 1.2, 0.05)   # call MSD function directly
```

11.4 Headless / server environments

```
import matplotlib
matplotlib.use('Agg')   # set BEFORE importing whitenoise
import whitenoise as wn

# Now all plot functions work without a display; use save_path= to write files
wn.publish_msd(result, save_path='msd.png', show=False)
```

12. Full API Quick Reference

I/O

Function	Returns
`wn.read_csv(path)`	`(time, values, metadata)`
`wn.read_csv_multi(path)`	`list of (time, values, metadata)`
`wn.export_csv(result, path)`	Writes one-row CSV
`wn.export_summary(cr, path)`	Writes multi-row CSV from ComparisonResult

Core

Function	Returns
`wn.compute_msd(x)`	`(lags, msd)` arrays
`wn.fit_msd(lags, msd, model)`	`FitResult` or `None`
`wn.list_models()`	Prints table (no return value)
`wn.get_model(name)`	Registry dict with `'msd'`, `'pdf'`, `'params'`, `'status'`

Preprocessing

Function	Returns
`wn.detrend(values, method='linear')`	1D ndarray (residuals)
`wn.normalize(values, method='zscore')`	1D ndarray
`wn.smooth(values, window=5, method='moving_average')`	1D ndarray (same length)

Analysis

Function	Returns
`wn.analyze(source, model='cosine')`	`AnalysisResult`
`wn.compare(datasets, model='cosine')`	`ComparisonResult`
`wn.print_comparison(cr)`	Prints ASCII table
`wn.batch_analyze(source, model='cosine')`	`ComparisonResult`
`wn.batch_model_search(path, models=None)`	`ComparisonResult`

Exploratory plots (viz/explore.py)

Function	Description
`wn.plot_series(x, values, metadata=meta)`	Raw or preprocessed sequential data
`wn.plot_msd_empirical(lags, msd)`	Empirical MSD before fitting
`wn.plot_msd(result)`	Empirical scatter + fitted line
`wn.plot_pdf(result)`	Displacement histogram + Gaussian PDF
`wn.plot_timeseries(result)`	Preprocessed observable
`wn.plot_diagnostics(result)`	2×2 overview figure

Publication plots (viz/publish.py)

Function	Description
`wn.publish_msd(result, palette, save_path)`	High-quality MSD plot (serif, dpi=150)
`wn.publish_pdf(result, lag_index, save_path)`	High-quality displacement PDF
`wn.publish_comparison(cr, save_path)`	Horizontal μ bar chart with 95% CI

13. Validated Benchmarks

These values serve as reference checks when testing with published datasets:

Dataset	Model	Expected μ	Reference
Philippine earthquakes	exponential	1.00 – 1.19	Roque et al. (2024)
Solar sunspot number	exponential	≈ 1.15	Toledo et al. (2024)
GBR coral bleaching	cosine	≈ 4.64	Elnar et al. (2021)
CO&sub2; Keeling curve	cosine	0.91 – 0.97	Elnar et al. (2024)
X-ray binaries	cosine	[0.50, 1.39]	Calotes thesis (2024)

14. Known Limitations

- **9 stub models**: `exp_whittaker`, `bessel_K`, `hypergeom_F1`, `bessel_I`, `hypergeom_3F2`, `csc_power`, `cot_power`, `bessel_pair`, `bessel_pair2` raise `NotImplementedError`.
- **Parallel batch with arrays**: `n_jobs > 1` only works with CSV file paths. Pass `n_jobs=1` when sources are in-memory arrays.
- **Headless display**: On servers without a display, set `matplotlib.use('Agg')` before importing whitenoise, then use `show=False` and `save_path=` arguments.

- **MSD reliability**: Only lags up to N/2 are computed. Large-lag estimates (near N/2) have fewer averaging pairs and are noisier — consider using `max_lag_fraction=0.5` for fitting if the far end of the MSD is erratic.
- **Unweighted fitting (OLS, not WLS)**: The fitter minimises the unweighted sum of squared residuals. All lags receive equal weight regardless of how many data pairs contributed to each MSD estimate. Noisy large-lag points can therefore bias the fit and distort R^2. Weighted least squares (WLS) is not currently implemented — use `max_lag_fraction` to exclude unreliable large-lag values as a workaround.

15. Citation

If you use this package in published research, please cite:

```
Bernido, C. C., & Carpio-Bernido, M. V. (2015).
White Noise Analysis and Quantum Field Theory.
AIP Conference Proceedings, 1618.

Bernido, C. C., & Carpio-Bernido, M. V. (2015).
Modified diffusion processes with memory — Table 3.1.
Stochastic and Infinite Dimensional Analysis, Springer.
```

For the whitenoise software package:

```
Bernido Group. (2025). whitenoise: Stochastic White Noise Analysis (SWNA).
University of San Carlos, Philippines.
Version 0.1.0.
```